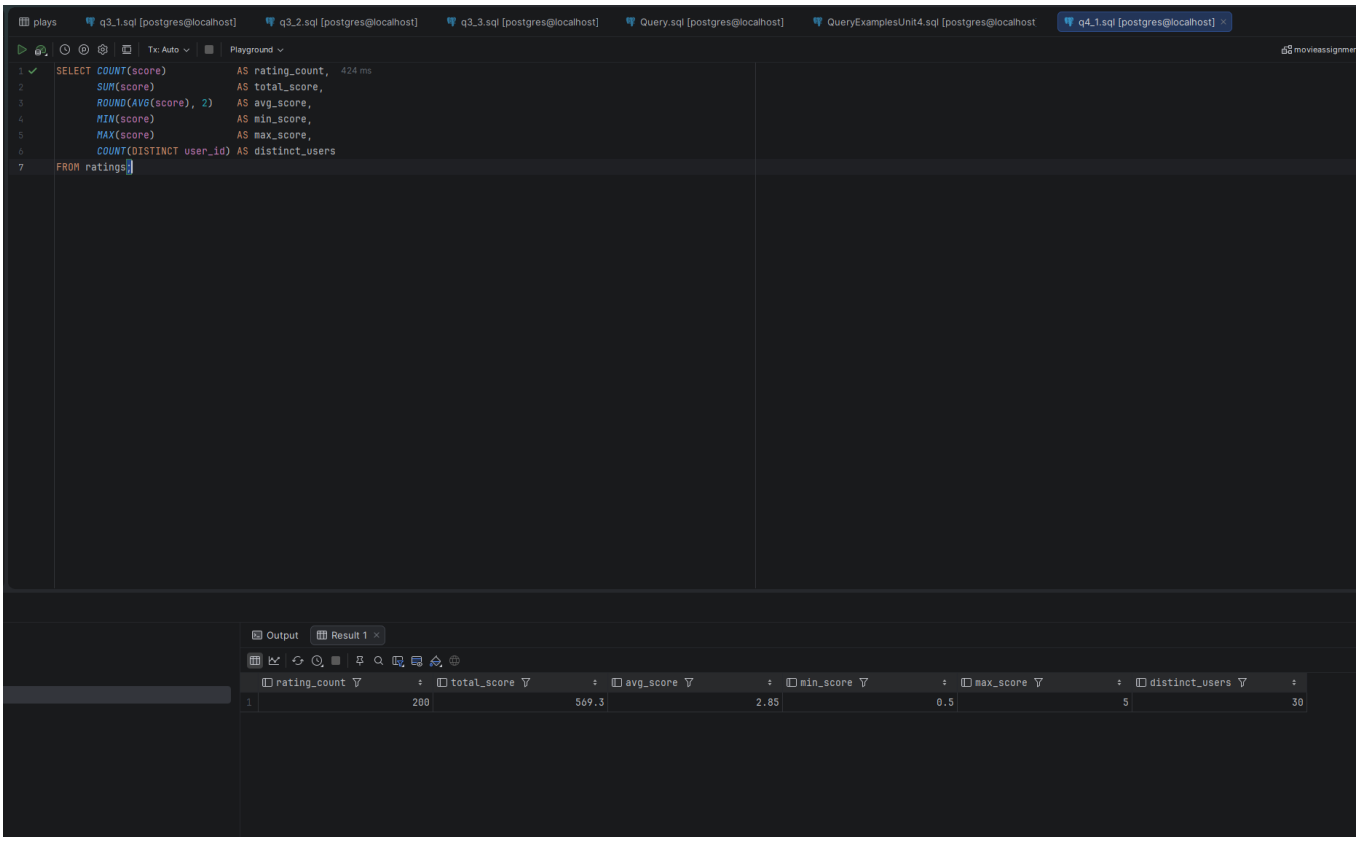
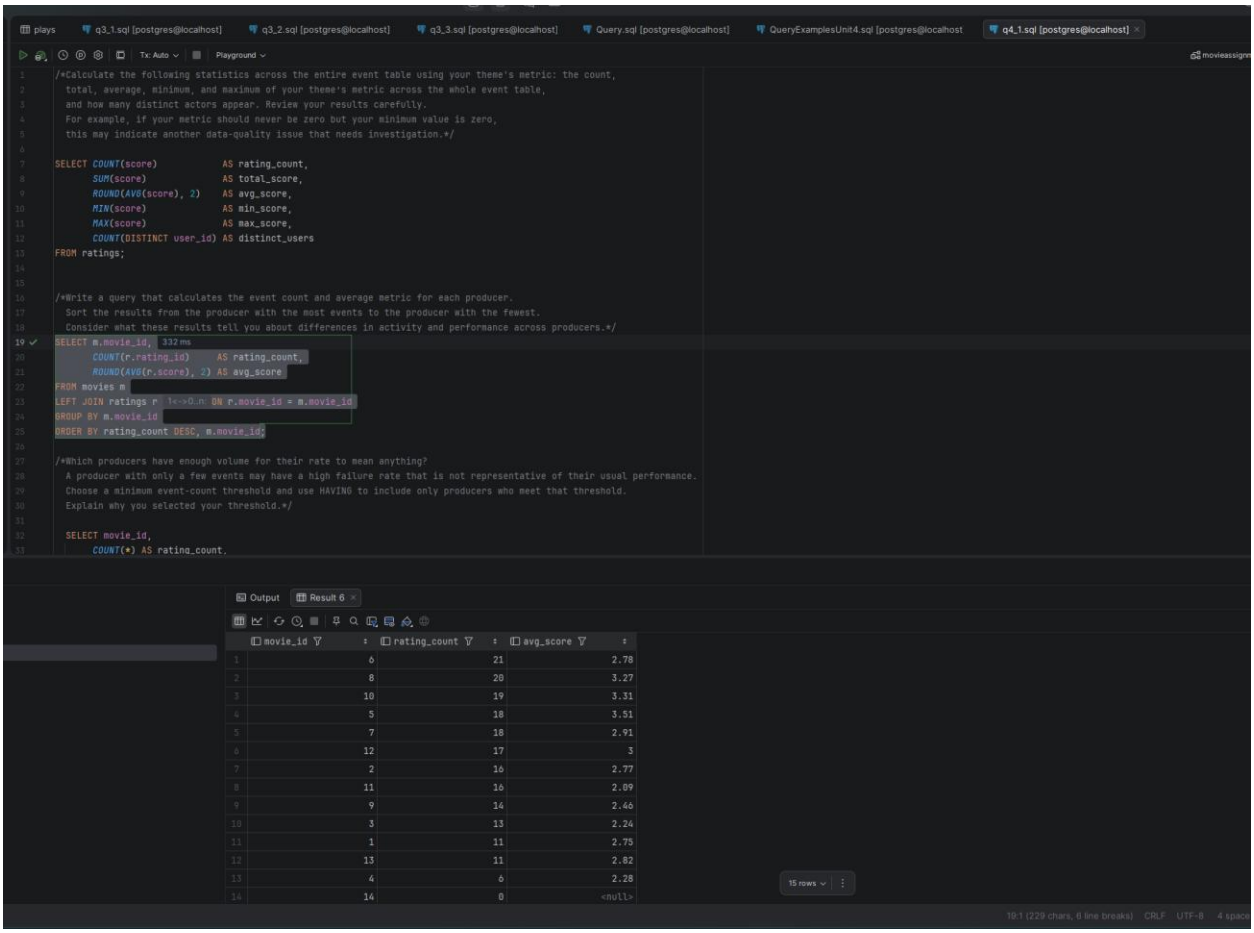
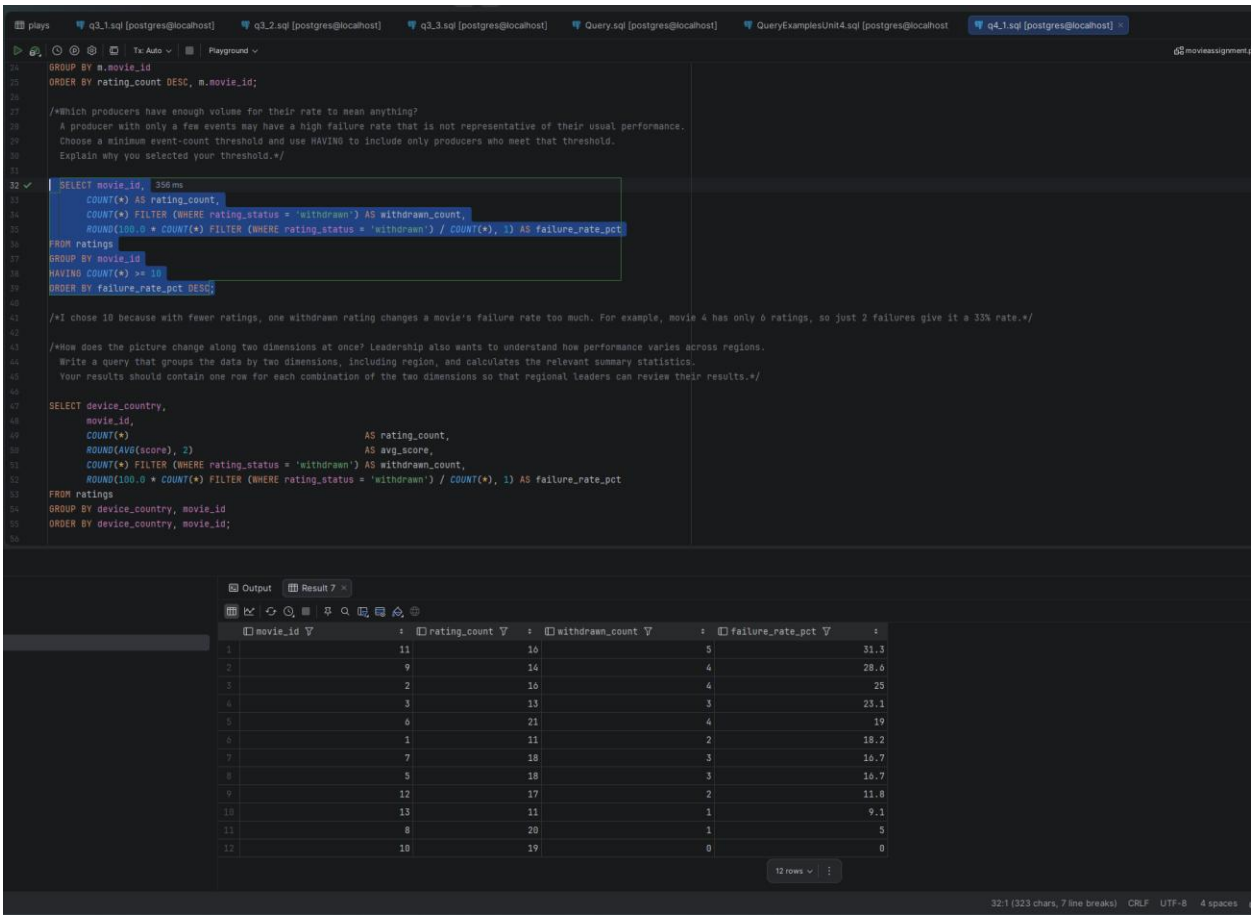
Question 1


plays
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
Query.sql [postgres@localhost]
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
Tx: Auto
Playground
movieassignmen
SELECT COUNT(score) AS rating_count, 424 ms
SUM(score) AS total_score,
ROUND(AVG(score), 2) AS avg_score,
MIN(score) AS min_score,
MAX(score) AS max_score,
COUNT(DISTINCT user_id) AS distinct_users
FROM ratings;
Output
Result 1
rating_count
total_score
avg_score
min_score
max_score
distinct_users
1
200
569.3
2.85
0.5
5
30

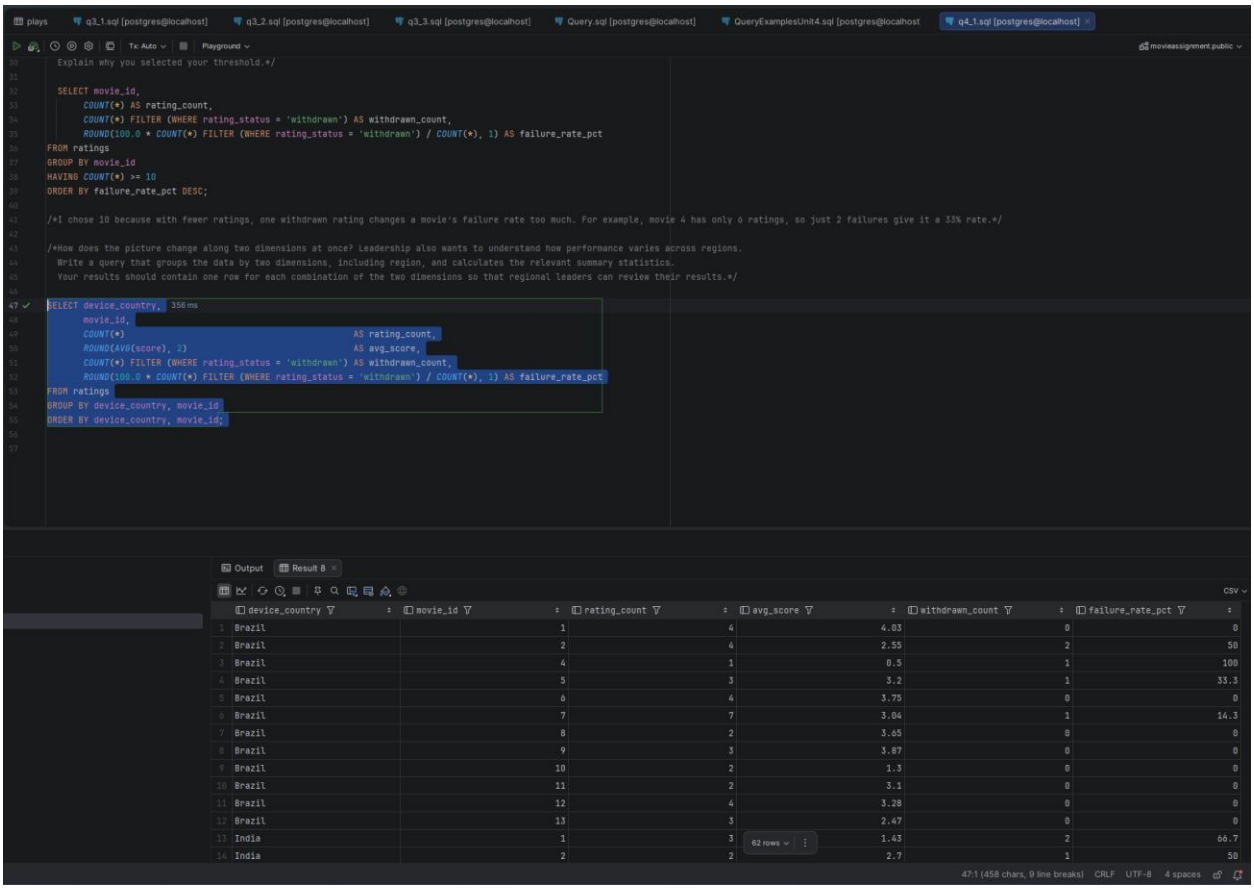
Question 2:


plays
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
Query.sql [postgres@localhost]
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
Tx: Auto
Playground
movieassignm
/*Calculate the following statistics across the entire event table using your theme's metric: the count,
total, average, minimum, and maximum of your theme's metric across the whole event table,
and how many distinct actors appear. Review your results carefully.
For example, if your metric should never be zero but your minimum value is zero,
this may indicate another data-quality issue that needs investigation.*/
SELECT COUNT(score) AS rating_count,
SUM(score) AS total_score,
ROUND(AVG(score), 2) AS avg_score,
MIN(score) AS min_score,
MAX(score) AS max_score,
COUNT(DISTINCT user_id) AS distinct_users
FROM ratings;
/*Write a query that calculates the event count and average metric for each producer.
Sort the results from the producer with the most events to the producer with the fewest.
Consider what these results tell you about differences in activity and performance across producers.*/
SELECT m.movie_id, 332 ms
COUNT(r.rating_id) AS rating_count,
ROUND(AVG(r.score), 2) AS avg_score
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id
GROUP BY m.movie_id
ORDER BY rating_count DESC, m.movie_id;
/*Which producers have enough volume for their rate to mean anything?
A producer with only a few events may have a high failure rate that is not representative of their usual performance.
Choose a minimum event-count threshold and use HAVING to include only producers who meet that threshold.
Explain why you selected your threshold.*/
SELECT movie_id,
COUNT(*) AS rating_count,
Output
Result 6
movie_id
rating_count
avg_score
1 6 21 2.78
2 8 20 3.27
3 10 19 3.31
4 5 18 3.51
5 7 18 2.91
6 12 17 3
7 2 16 2.77
8 11 16 2.09
9 9 14 2.46
10 3 13 2.24
11 1 11 2.75
12 13 11 2.82
13 4 6 2.28
14 14 0 <null>
15 rows
10:1 (229 chars, 6 line breaks)
CRLF
UTF-8
4 space

Question 3:


plays
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
Query.sql [postgres@localhost]
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
Tx: Auto
Playground
GROUP BY m.movie_id
ORDER BY rating_count DESC, m.movie_id;
/*Which producers have enough volume for their rate to mean anything?
A producer with only a few events may have a high failure rate that is not representative of their usual performance.
Choose a minimum event-count threshold and use HAVING to include only producers who meet that threshold.
Explain why you selected your threshold.*/
SELECT movie_id, 356 ms
COUNT(*) AS rating_count,
COUNT(*) FILTER (WHERE rating_status = 'withdrawn') AS withdrawn_count,
ROUND(100.0 * COUNT(*) FILTER (WHERE rating_status = 'withdrawn') / COUNT(*), 1) AS failure_rate_pct
FROM ratings
GROUP BY movie_id
HAVING COUNT(*) >= 10
ORDER BY failure_rate_pct DESC;
/*I chose 10 because with fewer ratings, one withdrawn rating changes a movie's failure rate too much. For example, movie 4 has only 6 ratings, so just 2 failures give it a 33% rate.*/
/*How does the picture change along two dimensions at once? Leadership also wants to understand how performance varies across regions.
Write a query that groups the data by two dimensions, including region, and calculates the relevant summary statistics.
Your results should contain one row for each combination of the two dimensions so that regional leaders can review their results.*/
SELECT device_country,
movie_id,
COUNT(*) AS rating_count,
ROUND(AVG(score), 2) AS avg_score,
COUNT(*) FILTER (WHERE rating_status = 'withdrawn') AS withdrawn_count,
ROUND(100.0 * COUNT(*) FILTER (WHERE rating_status = 'withdrawn') / COUNT(*), 1) AS failure_rate_pct
FROM ratings
GROUP BY device_country, movie_id
ORDER BY device_country, movie_id;
Output
Result 7
movie_id
rating_count
withdrawn_count
failure_rate_pct
1 11 16 5 31.3
2 9 14 4 28.6
3 2 16 4 25
4 3 13 3 23.1
5 6 21 4 19
6 1 11 2 18.2
7 7 18 3 16.7
8 5 18 3 16.7
9 12 17 2 11.8
10 13 11 1 9.1
11 8 20 1 5
12 10 19 0 0
12 rows
32:1 (323 chars, 7 line breaks)
CRLF
UTF-8
4 spaces

Question 4:



```
Explain why you selected your threshold.*/

SELECT movie_id,
    COUNT(*) AS rating_count,
    COUNT(*) FILTER (WHERE rating_status = 'withdrawn') AS withdrawn_count,
    ROUND(100.0 * COUNT(*) FILTER (WHERE rating_status = 'withdrawn') / COUNT(*), 1) AS failure_rate_pct
FROM ratings
GROUP BY movie_id
HAVING COUNT(*) >= 10
ORDER BY failure_rate_pct DESC;

/*I chose 10 because with fewer ratings, one withdrawn rating changes a movie's failure rate too much. For example, movie 4 has only 6 ratings, so just 2 failures give it a 33% rate.*/

/*How does the picture change along two dimensions at once? Leadership also wants to understand how performance varies across regions.
  Write a query that groups the data by two dimensions, including region, and calculates the relevant summary statistics.
  Your results should contain one row for each combination of the two dimensions so that regional leaders can review their results.*/

SELECT device_country,
    movie_id,
    COUNT(*)                                                    AS rating_count,
    ROUND(AVG(score), 2)                                        AS avg_score,
    COUNT(*) FILTER (WHERE rating_status = 'withdrawn')         AS withdrawn_count,
    ROUND(100.0 * COUNT(*) FILTER (WHERE rating_status = 'withdrawn') / COUNT(*), 1) AS failure_rate_pct
FROM ratings
GROUP BY device_country, movie_id
ORDER BY device_country, movie_id;
```

356 ms

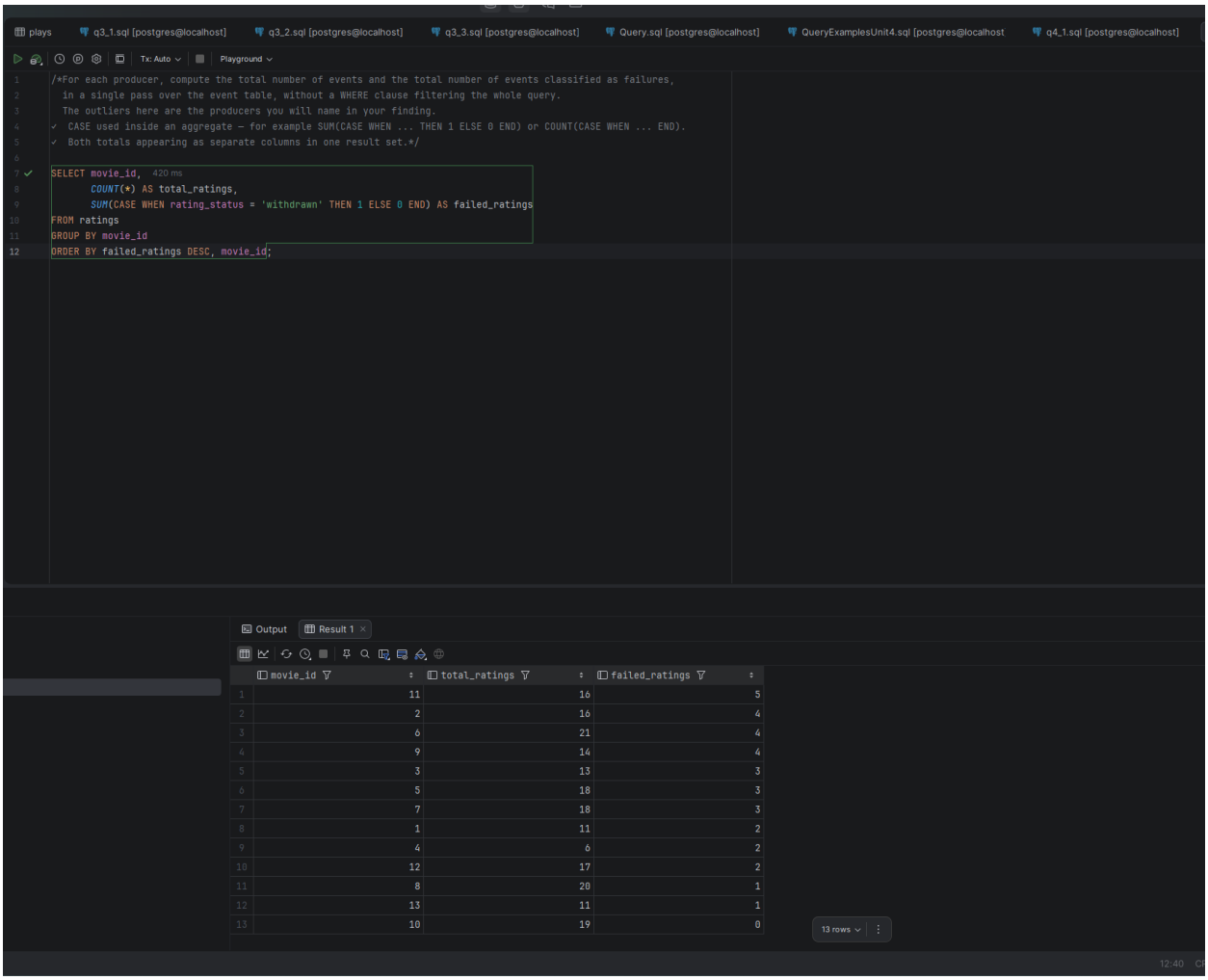
Output | Result 8

	device_country	movie_id	rating_count	avg_score	withdrawn_count	failure_rate_pct
1	Brazil	1	4	4.03	0	0
2	Brazil	2	4	2.55	2	50
3	Brazil	4	1	0.5	1	100
4	Brazil	5	3	3.2	1	33.3
5	Brazil	6	4	3.75	0	0
6	Brazil	7	7	3.04	1	14.3
7	Brazil	8	2	3.65	0	0
8	Brazil	9	3	3.87	0	0
9	Brazil	10	2	1.3	0	0
10	Brazil	11	2	3.1	0	0
11	Brazil	12	4	3.28	0	0
12	Brazil	13	3	2.47	0	0
13	India	1	3	1.43	2	66.7
14	India	2	2	2.7	1	50

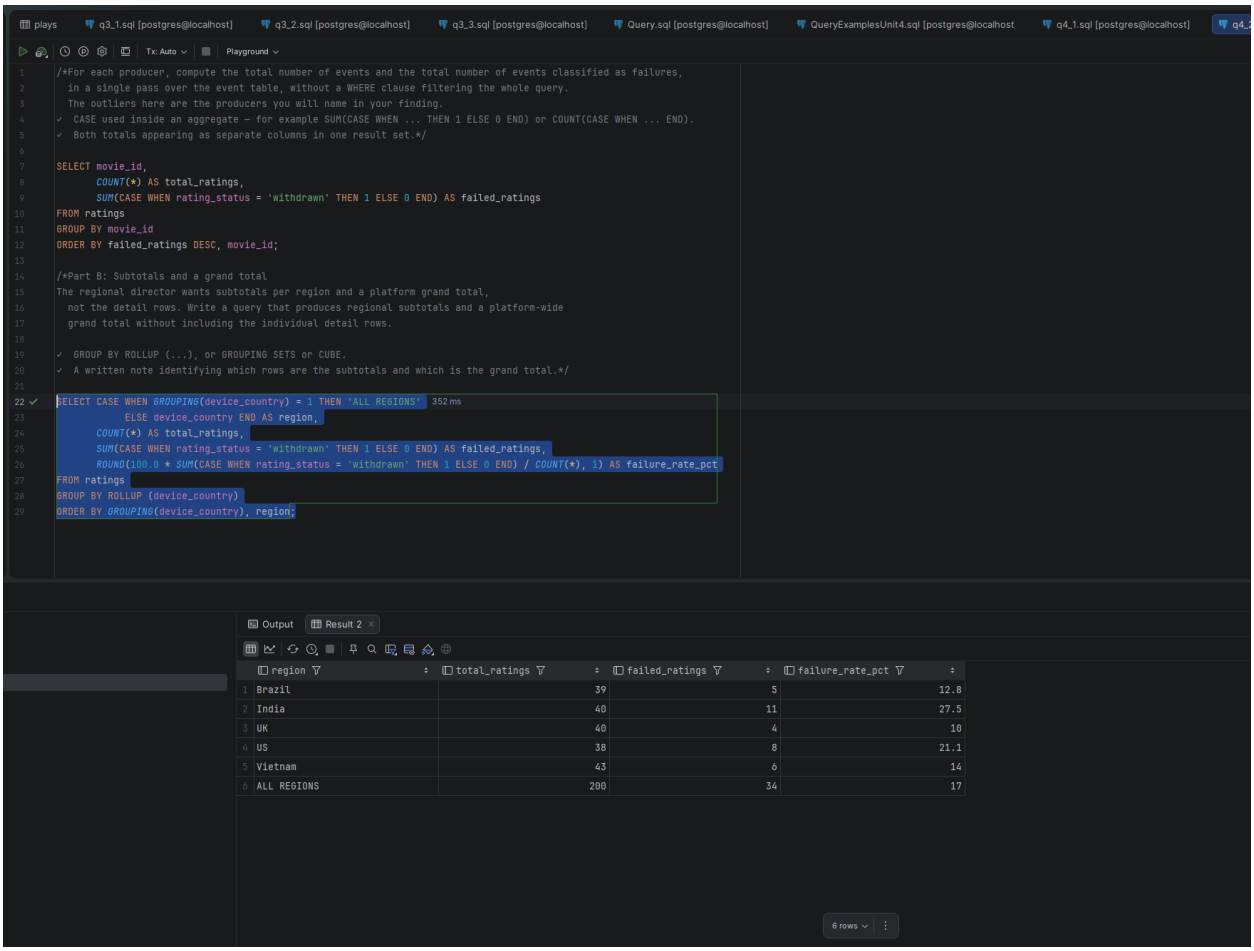
62 rows

47:1 (458 chars, 9 line breaks) CRLF UTF-8 4 spaces

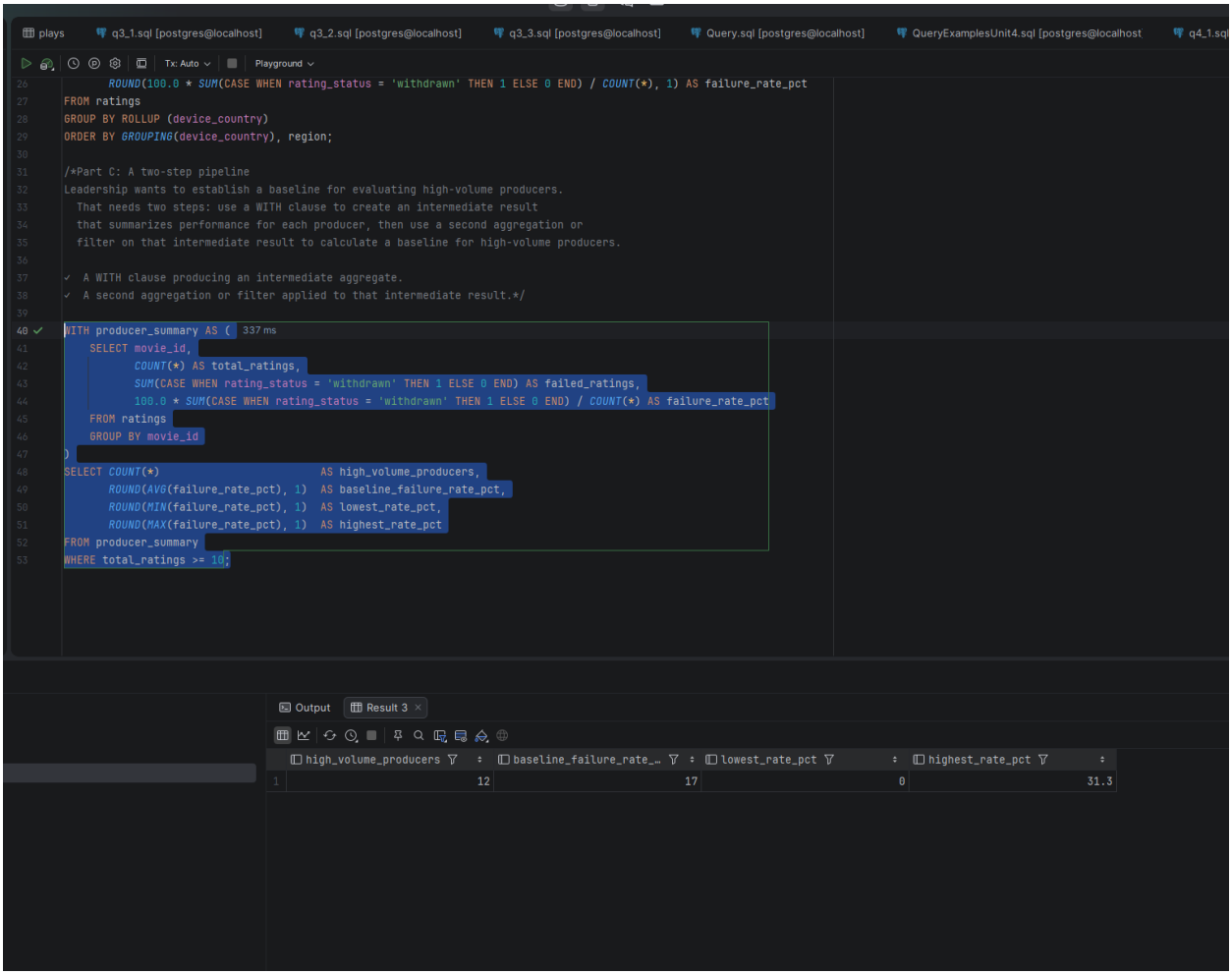
Part A


plays
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
Query.sql [postgres@localhost]
QueryExamplesUnit4.sql [postgres@localhost]
q4_1.sql [postgres@localhost]
Tx: Auto
Playground
/*For each producer, compute the total number of events and the total number of events classified as failures,
in a single pass over the event table, without a WHERE clause filtering the whole query.
The outliers here are the producers you will name in your finding.
CASE used inside an aggregate — for example SUM(CASE WHEN ... THEN 1 ELSE 0 END) or COUNT(CASE WHEN ... END).
Both totals appearing as separate columns in one result set.*/
SELECT movie_id, 420 ms
COUNT(*) AS total_ratings,
SUM(CASE WHEN rating_status = 'withdrawn' THEN 1 ELSE 0 END) AS failed_ratings
FROM ratings
GROUP BY movie_id
ORDER BY failed_ratings DESC, movie_id;
Output
Result 1
movie_id
total_ratings
failed_ratings
1 11 16 5
2 2 16 4
3 6 21 4
4 9 14 4
5 3 13 3
6 5 18 3
7 7 18 3
8 1 11 2
9 4 6 2
10 12 17 2
11 8 20 1
12 13 11 1
13 10 19 0
13 rows
12:40

Part B:


plays
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
Query.sql [postgres@localhost]
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
Tx: Auto
Playground
/*For each producer, compute the total number of events and the total number of events classified as failures,
in a single pass over the event table, without a WHERE clause filtering the whole query.
The outliers here are the producers you will name in your finding.
✓ CASE used inside an aggregate — for example SUM(CASE WHEN ... THEN 1 ELSE 0 END) or COUNT(CASE WHEN ... END).
✓ Both totals appearing as separate columns in one result set.*/
SELECT movie_id,
COUNT(*) AS total_ratings,
SUM(CASE WHEN rating_status = 'withdrawn' THEN 1 ELSE 0 END) AS failed_ratings
FROM ratings
GROUP BY movie_id
ORDER BY failed_ratings DESC, movie_id;
/*Part B: Subtotals and a grand total
The regional director wants subtotals per region and a platform grand total,
not the detail rows. Write a query that produces regional subtotals and a platform-wide
grand total without including the individual detail rows.
✓ GROUP BY ROLLUP (...), or GROUPING SETS or CUBE.
✓ A written note identifying which rows are the subtotals and which is the grand total.*/
SELECT CASE WHEN GROUPING(device_country) = 1 THEN 'ALL REGIONS' 352 ms
ELSE device_country END AS region,
COUNT(*) AS total_ratings,
SUM(CASE WHEN rating_status = 'withdrawn' THEN 1 ELSE 0 END) AS failed_ratings,
ROUND(100.0 * SUM(CASE WHEN rating_status = 'withdrawn' THEN 1 ELSE 0 END) / COUNT(*), 1) AS failure_rate_pct
FROM ratings
GROUP BY ROLLUP (device_country)
ORDER BY GROUPING(device_country), region;
Output
Result 2
region
total_ratings
failed_ratings
failure_rate_pct
1 Brazil 39 5 12.8
2 India 40 11 27.5
3 UK 40 4 10
4 US 38 8 21.1
5 Vietnam 43 6 14
6 ALL REGIONS 200 34 17
6 rows

Part C:


plays
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
Query.sql [postgres@localhost]
QueryExamplesUnit4.sql [postgres@localhost
Tx: Auto
Playground
ROUND(100.0 * SUM(CASE WHEN rating_status = 'withdrawn' THEN 1 ELSE 0 END) / COUNT(*), 1) AS failure_rate_pct
FROM ratings
GROUP BY ROLLUP (device_country)
ORDER BY GROUPING(device_country), region;
/*Part C: A two-step pipeline
Leadership wants to establish a baseline for evaluating high-volume producers.
That needs two steps: use a WITH clause to create an intermediate result
that summarizes performance for each producer, then use a second aggregation or
filter on that intermediate result to calculate a baseline for high-volume producers.
✓ A WITH clause producing an intermediate aggregate.
✓ A second aggregation or filter applied to that intermediate result.*/
WITH producer_summary AS (337 ms
SELECT movie_id,
COUNT(*) AS total_ratings,
SUM(CASE WHEN rating_status = 'withdrawn' THEN 1 ELSE 0 END) AS failed_ratings,
100.0 * SUM(CASE WHEN rating_status = 'withdrawn' THEN 1 ELSE 0 END) / COUNT(*) AS failure_rate_pct
FROM ratings
GROUP BY movie_id
)
SELECT COUNT(*) AS high_volume_producers,
ROUND(AVG(failure_rate_pct), 1) AS baseline_failure_rate_pct,
ROUND(MIN(failure_rate_pct), 1) AS lowest_rate_pct,
ROUND(MAX(failure_rate_pct), 1) AS highest_rate_pct
FROM producer_summary
WHERE total_ratings >= 10;
Output
Result 3
high_volume_producers
baseline_failure_rate_...
lowest_rate_pct
highest_rate_pct
1
12
17
0
31.3